The CNN lab: a convolutional network on CIFAR-10, same loop

AI Methods for Science (77222) · Part I, week 2; continues `lab_mlp.pdf`

The setup

`cnn_lab.py` (next to `mlp_lab2.py`, which it imports) has the same `train(config)` interface and the same result format. Only the ansatz changed: a stack of blocks (conv $3 \times 3 \to$ BatchNorm $\to$ ReLU, then 2×2 max-pooling), global average pooling, one linear layer. The defaults are Monday's recipe applied to a small CNN. The new keys:

key	default	meaning
`channels`	`[32, 64, 128]`	output channels of each block; `[` `lab_mlp.pdf` (`hidden [512, 25`
`convs_per_block`	`1`	`2` puts two conv layers in every b
`kernel`	`3`	size of every conv kernel
`batchnorm`	`true`	BatchNorm after every conv
`shuffle_pixels`	`false`	one fixed random permutation o every image, training and valida
`optimizer`, `lr`, `weight_decay`, `schedule`, `augment`, ...	AdamW, 10^{-3}, 0.05, cosine, true	as in `mlp_lab2.py`; defaults are
`epochs`	`10`	
`device`	`"auto"`	GPU (`cuda`) or Apple GPU (`mps`

Compute

A conv net costs more per epoch than the MLP. Default network, 352 steps: ≈ 6 s per epoch on an Apple M-series GPU, ≈ 20 s on an M5 CPU, 40–60 s on an older laptop CPU. **On a CPU, run 5 epochs instead of 10 and say so.** Do E16 first; if time is short, skip to E19 and do E17/E18 at home. Rules as before: do not edit the scripts; predict before each run; the test set once, in E19.

The experiments

E16. A CNN against the MLP, and what it is that it knows

~4 min on a GPU

Run. 10 epochs each: (a) the default CNN; (b) `channels` = `[]`, the MLP, under the same recipe; (c) the CNN with `shuffle_pixels` = `true`; (d) the MLP with `shuffle_pixels` = `true`.

Collect. validation accuracy vs. epoch, four runs on one axis; a table with `n_params`, `val_acc_at_best`, `sec_per_epoch`.

Answer. Before running: which of the four will be hurt by the shuffle, and by how much? Afterwards: the CNN has 18× fewer parameters than the MLP. What does it know that the MLP does not? Why is the MLP unhurt by the shuffle? After the shuffle, is the CNN still better than the MLP, and why could it be?

E17. Width and depth of the CNN

~5 min on a GPU

Run. 10 epochs: `channels` $\in$ {`[8,16,32]`, `[16,32,64]`, `[32,64,128]` (E16a), `[64,128,256]`}; if you have a GPU, also `[64,128,256]` with `convs_per_block` = 2.

Collect. `val_acc_at_best` vs. `n_params` ($\log x$), with E16's MLP as one more point; `sec_per_epoch` in the table.

Answer. How many parameters does a CNN need to beat the 1.7M-parameter MLP? Is the gain per parameter still rising at the top of your range? Where is the time going as the channels double (count the multiply-adds of one conv layer: $k^2\, C_{\text{in}}\, C_{\text{out}}\, H\, W$)?

E18. Take things away

~4 min on a GPU

Run. 10 epochs, from the default: `batchnorm` = `false`; `augment` = `false`; `optimizer` = `"sgd"` with `momentum` 0.9, `lr` = 0.05, `weight_decay` = 5×10^{-4} (the classic CNN recipe).

Collect. training and validation accuracy vs. epoch, the three and the default on one axis.

Answer. Which removal costs the most? Without augmentation, does the CNN overfit the way the MLP did in E4 (training 100%, validation loss rising)? Without BatchNorm, what happens in the first epochs, and does it recover? Does the classic SGD recipe match AdamW at 10 epochs?

E19. Your best CNN, test once

30 epochs; at home on a CPU

Run. the best architecture of E17 for 30 epochs. Choose by validation; then one run with `test` = `true`.

Collect. training and validation accuracy and validation loss vs. epoch; the table row; the test number.

Answer. Where does the validation loss bottom out now, and does early stopping still matter with cosine + augmentation? How far is your CNN from the best MLP of E15? Humans are at $\approx 94\%$ on CIFAR-10 and the best published networks above 99%: name two things you would change next.

The report

E16–E19 go into the same kind of report as E0–E15 (see `lab_mlp.pdf`): prediction, table, figures, a few sentences each, one PDF per group, submitted as a GitHub issue on the lab repository (*New issue* → *Lab report*). Which experiments are done in class and which at home is announced in class. Our reference curves are shown next to yours.